

Alla Vostra Mobile App Work Log - September 9, 2026

Compiled summary for Wednesday, September 9, 2026 - focus: verified U.S. Eastern-time date handling, full pre-existing worklog review, project-context refresh, snapshot documentation, iPhone native dev-client troubleshooting over hotspot and LAN/tunnel choices, Android Expo Go tunnel QR setup and cleanup, Metro/server lifecycle management, USB hub and cable data checks, Galaxy A3 direct USB confirmation, S20 Ultra and iPhone 14 hardware-capability interpretation, WebP image-load analysis, and current documentation artifact generation for Alla Vostra.

Generated in continuity with the established Alla Vostra worklog cadence: compact opening summary, source-material review, baseline, goals, grouped work-completed sections, validation, files touched, external account/device context, follow-up context, and output-artifact close.

This artifact is named `alla_vostra_worklog_september9.pdf` because the verified repository U.S. Eastern-time date used for generation was Wednesday, September 9, 2026, and no earlier September 9 root-level Alla Vostra worklog PDF existed before this documentation pass. The Eastern date was checked with `TZ=America/New_York` date, which reported Wednesday, September 9, 2026 in EDT even though UTC/system-facing contexts had already crossed into September 10. This PDF records the work window after `alla_vostra_worklog_september8.pdf` and documentation commit `daedbbd`, through synced snapshot commit `1b4094f` Updated project snapshots after native iOS testing, plus the temporary local Expo, QR, USB, and device-testing operations completed after that commit.

Source Material Reviewed

- Reviewed all 62 pre-existing root-level Alla Vostra PDF worklogs from May, June, July, August, September 7 Part 1, and September 8 to preserve the project naming pattern, letter-size PDF output, title cadence, metadata, footer language, section order, validation style, files-touched inventory, external-device notes, follow-up cadence, and output-artifact close.
- Extracted the full worklog set with `pdftotext` into a temporary text set and confirmed the study pass covered 62 text artifacts, 17,957 extracted lines, and 133,192 words of prior project history.
- Used `alla_vostra_worklog_september8.pdf` as the closest style anchor because it is the immediately preceding project PDF and uses the current long-form mobile-device testing, local build, and documentation shape.
- Rechecked `alla_vostra_worklog_september7_part1.pdf` because the September 9 testing depended on the Android Expo Go SDK 54 lane, WebP asset migration, native Stripe shim, iPhone development-build setup, and hardware/cable guidance established there.
- Reviewed the refreshed project snapshots generated earlier in the September 9 session with `read_project.sh`, including the mobile/app and web/desktop snapshot structure and the latest iOS generated-project tree captured by the snapshot refresh.
- Reviewed git history after the September 8 worklog, including commit `1b4094f`, which refreshed the app and web project snapshots and preserved the generated project-state documentation without adding runtime app behavior changes.
- Checked current repository status before this PDF generation pass and confirmed main was aligned with origin/main with no tracked working-tree changes before the new September 9 worklog artifact was created.
- Reviewed terminal results from Metro, Expo Go, dev-client, `devicectl`, `xcrun`, `adb`, `system_profiler`, `ioreg`, `lsdiff`, `curl`, `pdfinfo`, and `pdftotext` so the worklog records

concrete local tool state rather than only conversational notes.

Baseline Status Before September 9

- The September 8 worklog left the project with a successful free-account local iPhone development build path, CocoaPods installed, Xcode/iOS platform setup repaired, Apple Pay disabled for local development, and a documented temporary Stripe iOS header workaround inside ignored node_modules.
- The latest durable source work before the September 9 testing session had already prepared Expo SDK 54, expo-dev-client, Stripe native/Expo Go shimming, WebP runtime assets, Metro WebP resolution, and iOS bundle metadata for local native testing.
- The user wanted to continue hands-on multi-device testing with one iPhone and two Android phones, including Galaxy A3 and Galaxy S20 Ultra comparison, while using a mixture of hotspot, Expo tunnel, direct USB, and hub/cable arrangements.
- The project remained in a phase where Expo Go on Android is useful for visual/layout review, but native checkout and wallet behavior still require development builds or installed store/TestFlight builds because the real payment stack depends on native modules and server-backed payment paths.
- The WebP asset migration had reduced storage and transfer size, but previous Android testing still showed delayed image appearance, especially on older hardware and when loading through Expo Go/tunnel in development mode.
- The generated iOS native project under mob/ios and the patched Stripe header under mob/node_modules remained local/generated or ignored development state, not durable source files intended for manual editing unless a later patch-package or dependency fix is chosen.
- The hardware setup was still uncertain: some cables and the hub were not yet proven for data, and Xcode device visibility could reflect wireless pairing rather than a confirmed USB data connection.
- The repository was clean and synced at the start of the final PDF generation pass after the user committed the refreshed snapshots as 1b4094f.

Goals for September 9

- Reread the project context, snapshots, and full pre-existing worklog set so the session could continue from the real Alla Vostra history rather than a narrow recent-memory slice.
- Refresh and commit project snapshots after native iOS testing so the repository memory captured the generated iOS project tree, CocoaPods files, latest worklog file tree, and current app/web state.
- Help the user run Alla Vostra on a physical iPhone as a native development build while accounting for mobile hotspot, LAN, tunnel, and dev-client launch differences.
- Help the user run Alla Vostra on two Android phones through Expo Go, generate QR codes for the active tunnel, diagnose Expo Go errors, and cleanly stop the tunnels afterward.
- Clarify what internet-tunnel testing means for performance, responsiveness, image loading, and representativeness compared with direct USB, LAN, emulator/simulator, native dev build, and Play-installed app testing.
- Test random cables and hub paths pragmatically by checking whether macOS, Xcode, and ADB can see the devices, then separate phone-cable capability from hub-upstream capability.
- Explain the negotiated USB speed results for Galaxy A3, expected S20 Ultra and iPhone 14 capabilities, and whether physical-device testing remains more representative than emulators despite slower cable/reload paths.
- Analyze why images may still not appear immediately even after WebP conversion, focusing

on decode cost, preload behavior, dev-mode overhead, tunnel behavior, and active asset sizes.

Work Completed - Project Context and Snapshot Documentation

- Ran the project-reading workflow at the user request, using `read_project.sh` to regenerate `alla_vostra_APP_PROJECT_SNAPSHOT.txt` and `alla_vostra_WEB_PROJECT_SNAPSHOT.txt` at the repository root.
- Read the generated snapshots and the pre-existing PDF worklogs to rebuild the current project map: static web legacy surface under other, Expo Router mobile app under mob, serverless checkout/contact backend under server, and local/generated native testing artifacts under ignored mobile build paths.
- Confirmed the app snapshot now includes the generated local iOS native project and CocoaPods tree because `read_project.sh` does not exclude `mob/ios`, even though `mob/ios` remains generated local state rather than primary hand-authored source.
- Summarized the project to the user-ready state only after reviewing the mobile architecture, checkout stack, AppHeader/navigation behavior, startup tutorial, WebP migration, Stripe/PayPal/Postmark backend, Play Store release history, and September 8 iPhone build flow.
- Inspected git status after the snapshot refresh and saw only the two generated snapshot files as changed, with the temporary QR files not staged or committed.
- Recommended a commit message for the snapshot update with one header and hyphen-prefixed points, matching the user requested commit-message style.
- Confirmed the user committed the snapshot refresh as `1b4094f Updated project snapshots after native iOS testing`, with 21,731 insertions across the refreshed app and web snapshot documentation.
- Recorded that commit `1b4094f` was documentation-only: it refreshed project-state snapshots, updated the web snapshot timestamp and September 8 worklog listing, and did not add runtime app behavior changes.

Work Completed - iPhone Native Dev-Client Guidance

- Verified that the mobile package includes `expo-dev-client` and Expo SDK 54, making `npx expo start --dev-client` the appropriate server mode for the installed Alla Vostra development build.
- Explained the mobile-hotspot networking implication: `--host lan` works only when the iPhone can reach the Mac on the same local network, while `--host tunnel` is more reliable when the phone is on hotspot or local discovery is uncertain.
- Checked the Mac LAN address during testing and found `10.84.222.53` on `en0`, then confirmed Metro was reachable locally at port `8081` when the iOS dev server was alive.
- Used `xcrun devicectl list devices` to confirm Martin's iPhone moved from unavailable to connected/available as Xcode pairing recovered, while noting that Xcode visibility can also rely on paired/wireless state.
- Launched the installed Alla Vostra app on Martin's iPhone with `xcrun devicectl device process launch` and confirmed the command reported `com.allavostra.app` launched successfully.
- Tried both `allavostra://` and `exp+alla-vostra://` development-client payload URL launch forms against the local Metro URL to attach the installed iPhone development build to the dev server.
- Guided the user through the development-build launcher state: if Alla Vostra opens to the development build screen, select the visible development server or manually enter the current Metro URL.

- Later stopped the iOS/Metro development server at the user request so the hub/cable retesting could start from a clean network and process state.

Work Completed - Android Expo Go Tunnel Testing

- Generated a QR code for the initial exp.direct URL supplied by the user, exp://rzjbbc8-prahlin1-8081.exp.direct, then diagnosed the Expo Go "Something went wrong" result on both Android devices.
- Identified that the 8081 server was tied to an iOS native dev-client run rather than a clean Expo Go session, which made it a poor target for Android Expo Go QR testing.
- Started a clean Android Expo Go tunnel from mob with npx expo start --go --host tunnel --port 8082, which reported Tunnel connected, Tunnel ready, and Metro waiting on exp://rzjbbc8-prahlin1-8082.exp.direct.
- Generated and opened a new QR PNG for exp://rzjbbc8-prahlin1-8082.exp.direct, which the user confirmed worked on the Android devices.
- Watched Metro logs when the Android devices appeared to bundle for several minutes and confirmed the actual full Android bundle finished in roughly 5.7 seconds, with follow-up reload bundles completing in tens of milliseconds.
- Explained that if Expo Go still showed Bundling after Metro completed, the delay was likely tunnel/device/loading-state behavior rather than Metro still compiling the JavaScript bundle.
- Estimated the active Android development bundle at roughly 9.9 MB and explained why 10 Mbps download speed alone should not produce a five-minute bundle, while hotspot latency, Mac upload speed, exp.direct tunnel overhead, and two simultaneous devices could still create sluggish loading.
- Stopped the Android Expo Go tunnel on port 8082 at the user request and later confirmed no 8081/8082 listeners remained after shutdown.

Work Completed - Temporary QR Artifacts and Git Hygiene

- Generated temporary root-level QR images for the Expo tunnel session: android_expo_tunnel_qr.png and android_expo_go_8082_qr.png.
- Checked git status and git check-ignore for the QR PNGs and confirmed they were untracked but not ignored, meaning they were local-only until explicitly added, committed, and pushed.
- Explained that committing the tunnel QR images would be mildly undesirable because they expose temporary development-session URLs and clutter the repository, even though they did not contain passwords and the stopped tunnels should not remain useful.
- Clarified that Discard Changes in a Git UI would delete the untracked QR PNGs from the working directory, which was acceptable because the old tunnel sessions had already been shut down.
- Confirmed later that the QR PNGs were no longer present in the repository root and were not part of the snapshot commit or current PDF generation baseline.

Work Completed - USB Hub, Cable, and Device Capability Checks

- Tested the initial hub setup with system_profiler SPUSBHostDataType, system_profiler SPThunderboltDataType, ioreg -p IOUSB, xcrun devicectl, and adb devices -l, finding that macOS did not see the hub, either Android phone, or the iPhone as USB data devices.
- Detected one hub arrangement where the Mac drew AC power at about 8W, proving some charging/power path existed while still showing no USB data enumeration for the hub or attached phones.
- Explained that the no-data hub results pointed to the hub-to-Mac upstream connection, hub

- port choice, or hub cable mode before proving any individual phone cable bad.
- Tested one Samsung Android directly connected to the Mac with no hub and confirmed macOS saw SAMSUNG_Android, manufacturer SAMSUNG, serial 5203d5fe5b41c393, and Link Speed 480 Mb/s.
 - Clarified that the 480 Mb/s value was the negotiated speed of the Galaxy A3 direct-to-Mac connection using that cable, not an absolute statement about the cable maximum possible capability in every device/port combination.
 - Confirmed adb devices -l stayed empty for that direct Samsung connection, meaning USB data enumeration worked but ADB was not active or authorized; the phone still needed File transfer / Android Auto mode, Developer options, USB debugging, and the Allow USB debugging prompt.
 - Explained that the Galaxy S20 Ultra should be tested separately because negotiated speed is per phone, cable, hub, port, and mode; the S20 Ultra hardware is more capable than the A3 and can support faster USB/data and higher charging lanes with the right cable and charger.
 - Explained that the iPhone 14 remains USB 2.0-class over Lightning for wired data, with fast charging dependent on a proper USB-C power adapter/cable, and that Xcode paired visibility alone is not proof of current USB data enumeration.

Work Completed - Native Versus Emulator Testing Guidance

- Separated app reload/install convenience from representative runtime behavior: USB speed affects development iteration, install, reload, and log collection, but it does not make a physical device test less representative once the app is running.
- Explained that Galaxy A3 and iPhone 14 cannot match the S20 Ultra highest display refresh behavior because the A3 and iPhone 14 are effectively 60 Hz devices while the S20 Ultra can operate up to 120 Hz.
- Recommended setting the S20 Ultra to standard/60 Hz mode for apples-to-apples visual comparison across the A3, iPhone 14, and S20 Ultra.
- Confirmed that physical devices remain more representative than emulators for final judgment on touch feel, scrolling, keyboard behavior, safe areas, status/navigation bars, real network behavior, GPU/rendering quirks, memory pressure, and native payment/device APIs.
- Recommended emulators/simulators for fast layout checks, many screen sizes, repeatable debugging, and quick screenshots, with the physical A3, S20 Ultra, and iPhone 14 retained for final confidence checks.

Work Completed - WebP Image-Load Analysis

- Reviewed active image require paths across mob/app, mob/components, mob/data, mob/styles, and mob/utls to confirm the main runtime image references use WebP assets after the September 7 migration.
- Found that only a small number of images are explicitly prefetched, including the products cheeseboard hero in AppHeader and the tutorial Got It sprite in QuestionOverlay, while many Home, Products, About, Shop, and product-board images are still decoded on first render.
- Explained that WebP reduces file size and transfer/storage pressure, but the device still has to decompress each image into full bitmap memory, so old hardware such as the Galaxy A3 can still show delayed first paint for large source dimensions.
- Checked active and nearby asset sizes and noted that some WebP files remain in the multi-megabyte range, while several old original PNG/JPG files remain beside the WebPs intentionally because the user requested original images stay in place.

- Clarified that Expo Go and development mode add overhead through JavaScript bundling, source maps/dev tooling, tunnel delivery, and asset-serving behavior; production builds typically load and cache images more predictably.
- Identified the likely next optimization: preload first-screen hero/home/product images during the startup splash and consider lowering pixel dimensions for images that still decode to large bitmaps despite WebP compression.

Validation and Checks Performed

- Verified the U.S. Eastern-time date with TZ=America/New_York date, which reported Wednesday, September 9, 2026 at 22:40 EDT during the generation window.
- Extracted all 62 existing root-level worklog PDFs with pdftotext and confirmed the review set contained 17,957 lines and 133,192 words before drafting this artifact.
- Inspected September 8 and September 7 Part 1 worklog text and pdfinfo metadata to preserve the current eight-to-nine-page, letter-size, OpenAI Codex metadata, standard-library generator, and section-order conventions.
- Checked git status and git log before generation; main was aligned with origin/main at 1b4094f and had no tracked working-tree changes before this PDF was created.
- Checked lsof for ports 8081 and 8082 after tunnel shutdown and found no active listeners during the final generation baseline.
- Checked adb devices -l during the final baseline and found no currently attached/authorized Android devices after cable experiments had ended.
- Used system_profiler and ioreg during cable testing to confirm no hub/phone USB data enumeration through the hub, and to confirm the direct Galaxy A3 Samsung Android connection at 480 Mb/s when bypassing the hub.
- Reviewed image import paths and asset sizes with rg and find/ls to support the WebP image-load explanation with current repository evidence.
- Did not run a new Expo Go tunnel, iOS dev-client session, Android emulator, Gradle build, EAS build, production export, Stripe payment, PayPal capture, or Postmark email during this final PDF-generation pass.

Files Touched During the September 9 Window

- alla_vostra_APP_PROJECT_SNAPSHOT.txt - regenerated by read_project.sh and committed in 1b4094f, adding the generated iOS native project tree and CocoaPods file inventory into the app/mobile project snapshot.
- alla_vostra_WEB_PROJECT_SNAPSHOT.txt - regenerated by read_project.sh and committed in 1b4094f, refreshing the generated timestamp and adding the September 8 worklog to the project file tree.
- android_expo_tunnel_qr.png - temporarily generated at the repository root for the first Expo tunnel URL and later removed/discarded as an untracked local testing artifact.
- android_expo_go_8082_qr.png - temporarily generated at the repository root for the working Android Expo Go 8082 tunnel and later removed/discarded as an untracked local testing artifact.
- No app runtime source files were edited during this September 9 support/testing window; the source review and explanations focused on current files rather than code changes.
- No secrets, API keys, payment credentials, Apple account values, PayPal values, Stripe values, Vercel values, Postmark values, or device private credentials were added to source or written into this worklog.
- alla_vostra_worklog_september9.pdf - generated by this documentation pass as the latest root-level PDF worklog artifact for the project.

External Configuration and Device State

- The active iPhone testing device remained Martin's iPhone, an iPhone 14-class device with bundle identifier com.allavostra.app available from the local Alla Vostra development build installed during the September 8 setup flow.
- The iPhone testing discussion covered hotspot, LAN, tunnel, and dev-client launch behavior; successful testing depends on the iPhone opening the Alla Vostra development build and selecting or entering the current Metro URL.
- The Android testing devices were a Galaxy A3 and Galaxy S20 Ultra, both used for Expo Go visual testing through the working 8082 exp.direct tunnel during the session.
- The Android Expo Go tunnel used exp://rzjbbc8-prahlin1-8082.exp.direct while active; that tunnel was stopped before the final documentation pass and should not be treated as a persistent public endpoint.
- The Galaxy A3 direct USB test enumerated as SAMSUNG_Android at 480 Mb/s with serial 5203d5fe5b41c393, proving that at least one direct cable/device path supported USB data to the Mac.
- The hub-based phone setup did not enumerate as USB data in macOS during repeated checks, even when the Mac later showed low-wattage AC charging, so the hub/upstream path remains suspect for data debugging.
- The S20 Ultra and iPhone 14 hardware-capability notes were guidance for what those devices can support and how to test them; they were not recorded as confirmed negotiated USB results in the current cable arrangement.
- All Expo/Metro tunnels were stopped before the hub/cable restart plan, and no active ADB device remained visible at final baseline.

Follow-up Context for Future Sessions

- To restart Android Expo Go visual testing, run `cd mob && npx expo start --go --host tunnel --port 8082` or use `--host lan` when the Mac and Android phones are on the same reliable Wi-Fi network.
- To restart iPhone native dev-client testing, connect/unlock Martin's iPhone, confirm Xcode/devicectl sees it, run `cd mob && npx expo start --dev-client --host lan` when local network reachability is good, or use `--host tunnel` when hotspot routing makes LAN unreliable.
- If the iPhone opens only to the Alla Vostra development-build launcher, select the visible development server or manually enter the current Metro URL for that session.
- For faster and more representative Android iteration, test one Android device at a time when using the tunnel, prefer LAN over exp.direct when possible, and use native development or Play-installed builds for checkout/payment confidence.
- For cable testing, isolate one variable at a time: first plug one phone directly into the Mac to prove the phone cable, then add the hub, then add other phones, watching `system_profiler/ioreg` and `adb/devicectl` after each change.
- For the Galaxy A3, use it as the older/slow Android resilience target; for the S20 Ultra, use it as the modern Android target and set display motion smoothness to 60 Hz when comparing against the A3 and iPhone 14.
- For image-load improvement, consider adding a small startup asset-preload utility for first-screen hero/home/product assets and consider pixel-dimension reductions for still-large WebPs before judging perceived load time on the A3.
- For durable iOS native builds, remember that the local `StripeSwiftInterop.h` `node_modules` header patch may disappear after reinstall; convert it into a persistent patch or dependency update if the Xcode enum issue returns.
- Commit this September 9 PDF as a documentation-only artifact if following the recent

project cadence of source/config/snapshot commit first and PDF worklog commit second.

Output Artifact

- Created alla_vostra_worklog_september9.pdf at the repository root as the latest customary PDF worklog for the Alla Vostra project.
- Used the established root-level worklog filename pattern because this was the first September 9 worklog artifact, so no part suffix was required.
- Preserved the current letter-size PDF convention, OpenAI Codex metadata, Codex standard-library PDF generator creator/producer values, footer format, concise opening summary, section ordering, bullet-heavy documentation style, validation language, files-touched detail, external account/device context, follow-up section, and output-artifact close.
- This PDF records the September 9 support/testing work: verified Eastern-time date handling, full worklog study, project snapshot refresh and commit 1b4094f, iPhone dev-client network guidance, Android Expo Go tunnel QR repair, Expo/Metro shutdown, temporary QR cleanup, USB hub/cable diagnostics, Galaxy A3 data confirmation, S20 Ultra and iPhone 14 capability interpretation, emulator-versus-native testing guidance, WebP image-load analysis, and remaining follow-up risks.